

EMS API Gateway

Detailed Architecture, Ports, Routing and Request Flow

Purpose: Provide a single reference for understanding the current EMS Gateway architecture, the applications behind it, ports, explicit route predicates, JWT flow, CORS ownership, and troubleshooting.

Current verified architecture: API Gateway :8082, EMS Backend :8080, Auth Service :8083, React UI :3000. The Gateway is the browser-facing backend entry point.

1. Executive Architecture

The system separates authentication from EMS business functionality. Auth Service owns registration, credential verification and JWT generation. EMS Backend owns business APIs and validates/authORIZES JWTs locally. The Gateway routes requests by explicit API-resource path.

```
React UI :3000
|
v
API Gateway :8082
||
| /api/auth/** | /api/employees/**, /departments/**, /offices/**
v v
Auth Service :8083 EMS Backend :8080
||
v v
ems_auth_service javaguide_ems
```

2. Applications and Ports

Application	Port	Role	Gateway relationship
React UI	3000	Browser/client	Calls Gateway; not a Gateway destination
API Gateway	8082	Spring Cloud Gateway / WebFlux	Single browser-facing backend entry point
EMS Backend	8080	EMS business APIs + JWT validation/authorization	Gateway destination
Auth Service	8083	Registration/login + JWT generation	Gateway destination
Notification Service	Not specified in current Gateway design	Internal notification integration	Not routed by Gateway

Database ownership

Database	Owner	Contents
javaguide_ems	EMS Backend	Employee, Department, Office, Experience and related EMS domain data. Old users/roles/users_roles tables were removed.
ems_auth_service	Auth Service	users, roles, users_roles and authentication data.

3. Current Gateway Routing

The Gateway uses explicit resource-level Path predicates. There is no generic `/ems-dev/v1/**` catch-all route. This avoids overlap between authentication and EMS routes.

Route ID	Incoming path	Target
ems-auth-service-route	/ems-dev/v1/api/auth/**	http://localhost:8083
ems-employee-route	/ems-dev/v1/api/employees/**	http://localhost:8080
ems-department-route	/ems-dev/v1/api/departments/**	http://localhost:8080
ems-office-route	/ems-dev/v1/api/offices/**	http://localhost:8080

Gateway routing configuration

```
# Auth Service
spring.cloud.gateway.server.webflux.routes[0].id=ems-auth-service-route
spring.cloud.gateway.server.webflux.routes[0].uri=http://localhost:8083
spring.cloud.gateway.server.webflux.routes[0].predicates[0]=Path=/ems-dev/v1/api/auth/**

# Employee APIs
spring.cloud.gateway.server.webflux.routes[1].id=ems-employee-route
spring.cloud.gateway.server.webflux.routes[1].uri=http://localhost:8080
spring.cloud.gateway.server.webflux.routes[1].predicates[0]=Path=/ems-dev/v1/api/employees/**

# Department APIs
spring.cloud.gateway.server.webflux.routes[2].id=ems-department-route
spring.cloud.gateway.server.webflux.routes[2].uri=http://localhost:8080
spring.cloud.gateway.server.webflux.routes[2].predicates[0]=Path=/ems-dev/v1/api/departments/**

# Office APIs
spring.cloud.gateway.server.webflux.routes[3].id=ems-office-route
spring.cloud.gateway.server.webflux.routes[3].uri=http://localhost:8080
spring.cloud.gateway.server.webflux.routes[3].predicates[0]=Path=/ems-dev/v1/api/offices/**
```

Why explicit routes? The earlier `/ems-dev/v1/**` route also matched `/ems-dev/v1/api/auth/**`. That caused an Auth request to be sent to EMS and resulted in an authentication error. The explicit route design makes the service boundary unambiguous.

4. Authentication Flow

The Gateway does not authenticate the user. It only forwards the authentication request to Auth Service. Auth Service verifies credentials, loads the user's roles and creates the JWT.

```
POST /ems-dev/v1/api/auth/login
|
v
Gateway :8082
|
| Path matches /ems-dev/v1/api/auth/**
v
Auth Service :8083
|
| authenticate user
v
ems_auth_service
|
| roles + credentials
v
JWT generated by Auth Service
|
v
Response returns through Gateway :8082
```

Example request

```
POST http://localhost:8082/ems-dev/v1/api/auth/login
Content-Type: application/json
```

```
{
  "usernameOrEmail": "mohanlal",
  "password": "mohanlal"
}
```

Expected response contract

```
{
  "accessToken": "<JWT>",
  "tokenType": "Bearer",
  "role": "ROLE_USER"
}
```

5. Business API Flow with JWT

After login, the client sends the JWT as a Bearer token to the Gateway. The Gateway routes the request to EMS Backend. EMS validates the JWT locally and applies its Spring Security authorization rules.

```
GET /ems-dev/v1/api/employees/6
Authorization: Bearer <JWT>
|
v
Gateway :8082
|
| Path matches /ems-dev/v1/api/employees/**
v
EMS Backend :8080
|
+--> verify JWT signature
+--> check expiration
+--> extract username
+--> extract roles
+--> authorize HTTP operation
|
v
javaguide_ems
|
v
Employee response
|
v
Gateway :8082 -> Client
```

Important: EMS does not need to query `ems_auth_service` for every business request. Identity and role information are carried in the signed JWT.

6. JWT Responsibility Split

Responsibility	Auth Service	EMS Backend
Authenticate username/password	Yes	No
Generate/sign JWT	Yes	No
Put roles into JWT	Yes	No
Validate JWT signature	No	Yes
Check expiration	No	Yes
Extract username/roles	No	Yes
Authorize EMS resources	No	Yes

7. CORS Design

The Gateway is the browser-facing entry point, so CORS is configured there. Backend controller-level @CrossOrigin configuration was removed to prevent duplicate Access-Control-Allow-Origin headers.

```
spring.cloud.gateway.server.webflux.globalcors.add-to-simple-url-handler-mapping=true
spring.cloud.gateway.server.webflux.globalcors.cors-configurations.[/**].allowed-origins=http://localhost:3000
spring.cloud.gateway.server.webflux.globalcors.cors-configurations.[/**].allowed-methods=GET,POST,PUT,DELETE,OPTIONS
spring.cloud.gateway.server.webflux.globalcors.cors-configurations.[/**].allowed-headers=Authorization,Content-Type,Accept
```

Browser path

```
React :3000
|
| HTTP + Origin
v
Gateway :8082
|
+--> CORS policy
|
+--> route request
|
+--> Auth :8083
or
+--> EMS :8080
```

8. Verified End-to-End Tests

Verification	Result
Auth Service registration	Passed
Duplicate username/email checks	Passed at service logic level
Login with username/email	Passed
Invalid password rejection	Passed at service logic level
Gateway -> Auth registration	Passed
Gateway -> Auth login	Passed
Gateway -> EMS employee GET using Auth-generated JWT	Passed
ROLE_USER authorization through Gateway	Passed

9. Troubleshooting and Design Lessons

401 from Auth URL through Gateway: Check whether a broad `/ems-dev/v1/**` route is capturing the request. The current explicit Auth route should be `/ems-dev/v1/api/auth/** -> 8083`.

403 on login: First test directly against port 8083. Verify the test username/password pair and ensure Postman is not sending an inherited Bearer token. A wrong test password can look like a Gateway/security problem.

EMS 401: Check the Authorization header, JWT expiration, and the shared HMAC signing secret used by Auth and EMS.

EMS 403: Authentication succeeded but the user's role is not permitted for that HTTP operation. Check EMS SecurityConfig.

Browser CORS failure: Check Gateway CORS and ensure backend controllers do not add duplicate CORS headers.

10. Current Architecture Boundaries

API Gateway: routing and browser-facing CORS. Auth Service: registration, login, credential verification and JWT generation. EMS Backend: business APIs, JWT validation and authorization. Auth database: users/roles. EMS database: EMS domain data. Notification Service: internal notification integration and not currently exposed through the Gateway.

11. Next Integration Step

The backend Gateway architecture is now verified. The next step is to ensure the React frontend consistently calls the Gateway at port 8082 for authentication and EMS business APIs. This should be tested from the browser after the Postman-level routing tests are complete.

Future hardening, outside the current routing change: production secret management, asymmetric JWT signing/OIDC, service discovery, centralized configuration, timeouts/circuit breakers, distributed tracing, correlation IDs, centralized logging, and more formal API error responses.